

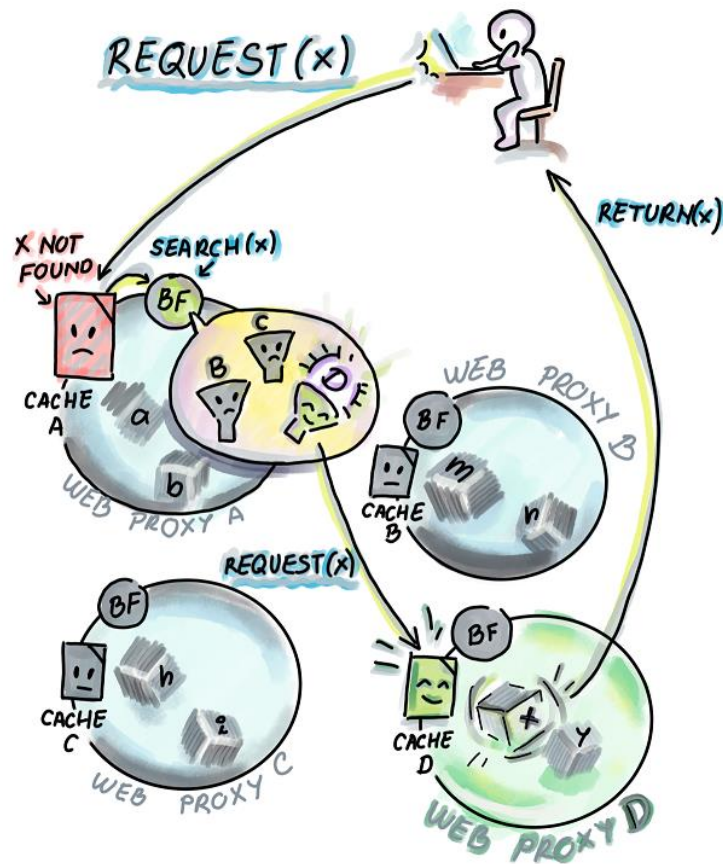
Data Stream II: Bloom Filter

COMPSCI 753

The University of Auckland

Outline

1. Sampling from a Data Stream
 - Sampling a fixed proportion
 - Sampling a fixed-size
2. Filtering a Data Stream: Bloom Filters
3. Finding Heavy Hitters in Data Stream
 - Misra-Gries Algorithm
 - CountMin Sketch Algorithm
 - Count Sketch Algorithm



Definitions

- Let U be a universe of size n
- Cash register model stream
 - Sequence of m elements $a_1, \dots, a_m$ where $a_i \in U$
 - Elements of U may or may not occur once or several times in the stream
 - E.g., A sequence of email addresses, a sequence of IP addresses,...
- Filtering a data stream
 - Select elements with property x from the stream

Filtering Data Stream

- Each element of data stream is a tuple
- Given a list of keys S , determine which stream elements are in S
- Obvious solution: Hash table
 - We **do not have enough memory to store all S** in a hash table
 - We might process millions of filters on the same stream

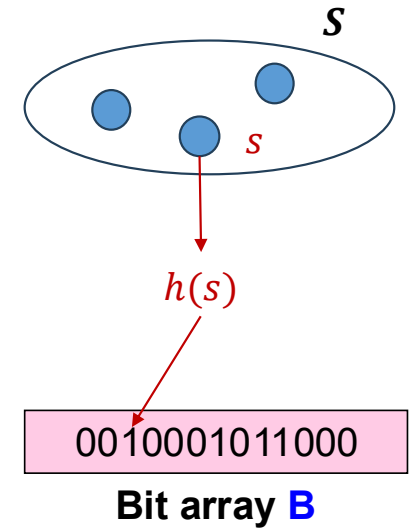
Applications: Email Spam Filtering

- We know 1 billion “good” email addresses
- If an email comes from one of these, it is NOT spam



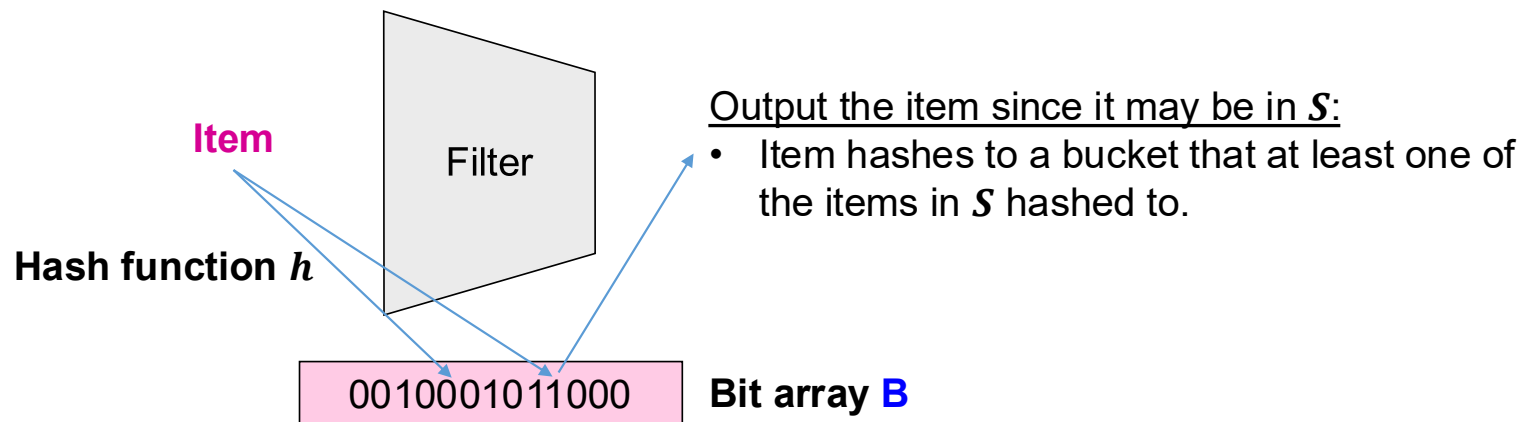
First Cut Solution

- Given a set of keys S
 - Create a bit array B of n bits, initially at 0 s
 - Choose a hash function h with range $[0, n)$
 - Hash each member $s \in S$ to one of n buckets, and set that bit to 1 , i.e., $B[h(s)] = 1$
- Hash each element a of the stream and output only those that hash to bit that was set to 1
 - Output a if $B[h(a)] = 1$



First Cut Solution

- Create **false positives** but **no false negatives**
 - If the item is in S , we surely output it (no false negatives)
 - If not, we may still output it (false positives)

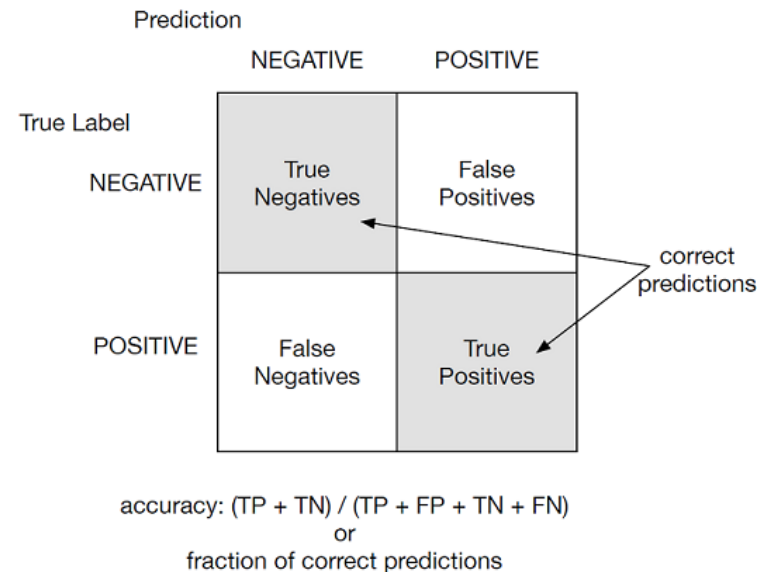


Drop the item:

- It hashes to a bucket set to 0 so it is surely not in S .

Parameter Settings

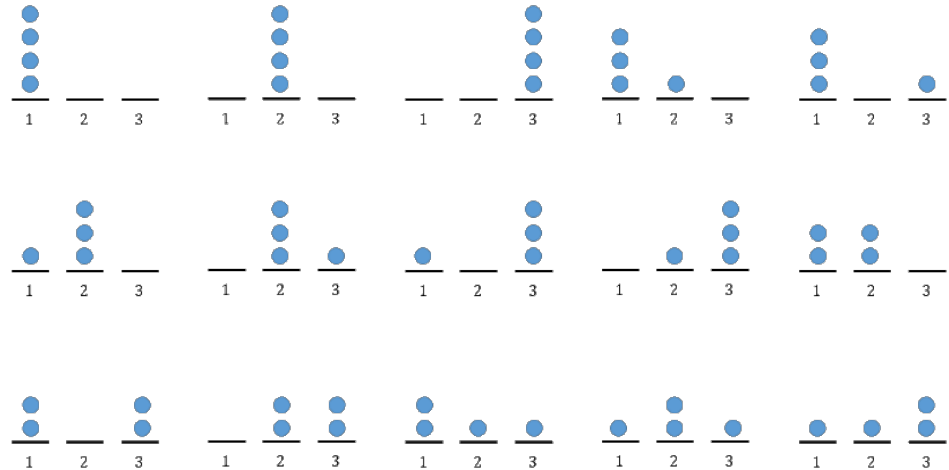
- $|S| = 1$ billion email addresses
 $|B| = 1\text{GB} = 8$ billion bits
- If the email address is in S , then it surely hashes to a bucket that has the bit set to **1**, so it always gets through (**no false negatives**)
- Approximately $1/8$ of the bits are set to **1**, so about $1/8$ of the addresses not in S get through to the output (**false positives**)
 - Less than $1/8$, because more than one address in S might hash to the same bit



Analysis: Balls and Bins Model

- We want to analyze the number of false positives
- General problem: If we randomly throw m balls into n bins, what is the probability that a bin get at least one ball?

- Our case:
 - Balls = elements of $\mathcal{S}$
 - Bins = buckets
 - Randomly throwing = hashing



Analysis: Balls and Bins Model

- We have m balls, n bins. **Throw the m balls independently**
- **Throwing one ball:**
 - Prob. that a bin has at least one ball: $\frac{1}{n}$
 - Prob. that a bin does not have any ball: $(1 - \frac{1}{n})$
- **Throwing m balls:**
 - Prob. that a bin does not have any ball: $(1 - \frac{1}{n})^m$
 - Prob. that a bin has at least one ball: $1 - (1 - \frac{1}{n})^m$

Analysis: Balls and Bins Model

- Given m balls and n bins, the probability that a bin does not have any ball is

$$\begin{aligned} & \left(1 - \frac{1}{n}\right)^m \\ &= \left(1 - \frac{1}{n}\right)^{n \times \frac{m}{n}} \longleftarrow n \rightarrow \infty, \left(1 - \frac{1}{n}\right)^n \sim \frac{1}{e} \\ &\sim (1/e)^{m/n} \end{aligned}$$

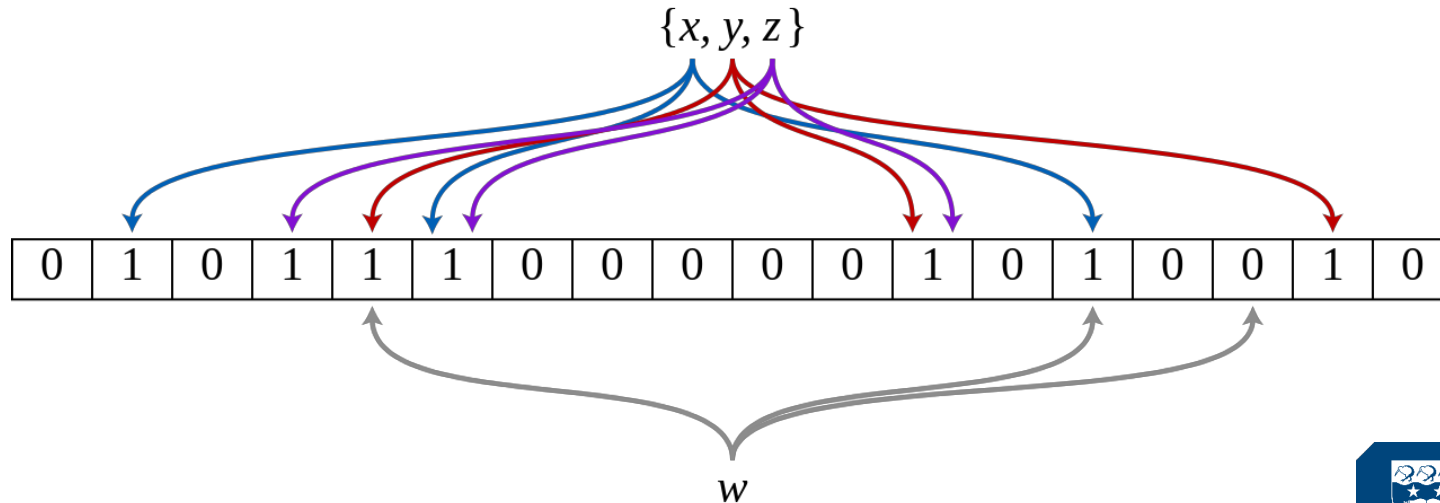
- Since $\left(1 - \frac{1}{n}\right)^m \sim e^{-m/n}$, the prob. that a bin gets at least one ball is: $1 - e^{-m/n}$
- Fraction of 1s in the array B = prob. of false positives = $1 - e^{-m/n}$
- Example: $m = 1$ billions, $n = 8$ billion bits = 1GB, the fraction of 1s in B is 0.1175 ($\sim 1/8 = 0.125$)

Bloom Filter

- $|S| = m, |B| = n$
- Use k independent hash function $h_1, \dots, h_k$ with range $[0, n)$.
- Construct Bloom filter:
 1. Set B to all 0 s.
 2. Hash each member $s \in S$ to one of n buckets using k hash functions, and set $B[h_i(s)] = 1$ for $i = 1, \dots, k$
- Filtering: When a stream element a arrives,
 - If $B[h_i(a)] = 1$ for all $i = 1, \dots, k$, then declare $a \in S$
 - Otherwise, discard the element a

Example: Bloom Filter

- $|S| = 3$, $|B| = 18$
- $S = \{x, y, z\}$
- $k = 3$
- Discard w since all $B[h(w)]$ are not all 1s.

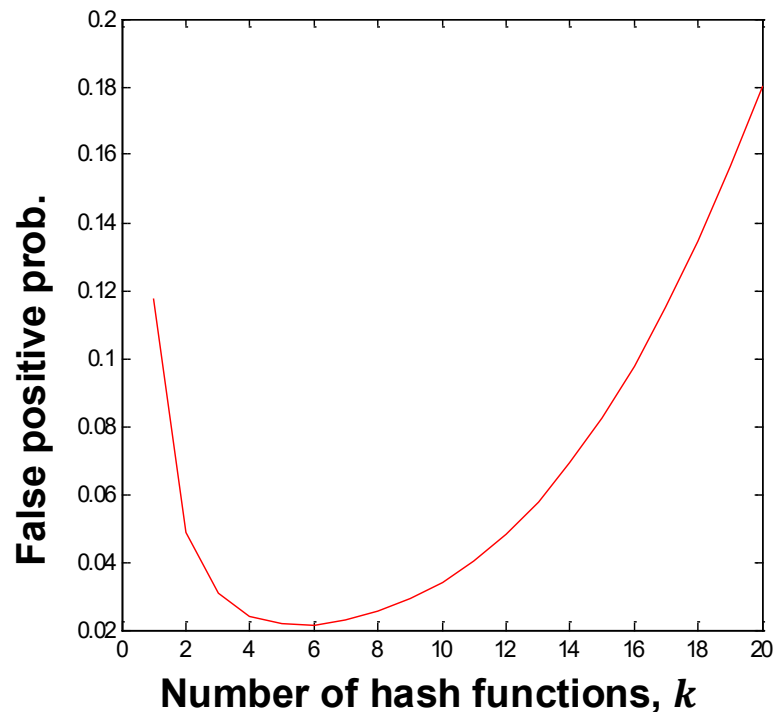


Analysis: Bloom Filter

- What fraction of the bit vector ***B*** are **1s**?
 - Randomly throw $k \times m$ balls into n bins.
 - So, fraction of **1s** is $1 - e^{-km/n}$
- Prob. that **1** position of ***a*** is **1s**: $1 - e^{-\frac{km}{n}}$
- Prob. that all ***k*** positions of ***a*** is **1s**: $\left(1 - e^{-\frac{km}{n}}\right)^k$
- So, **false positive** probability is: $\left(1 - e^{-\frac{km}{n}}\right)^k$

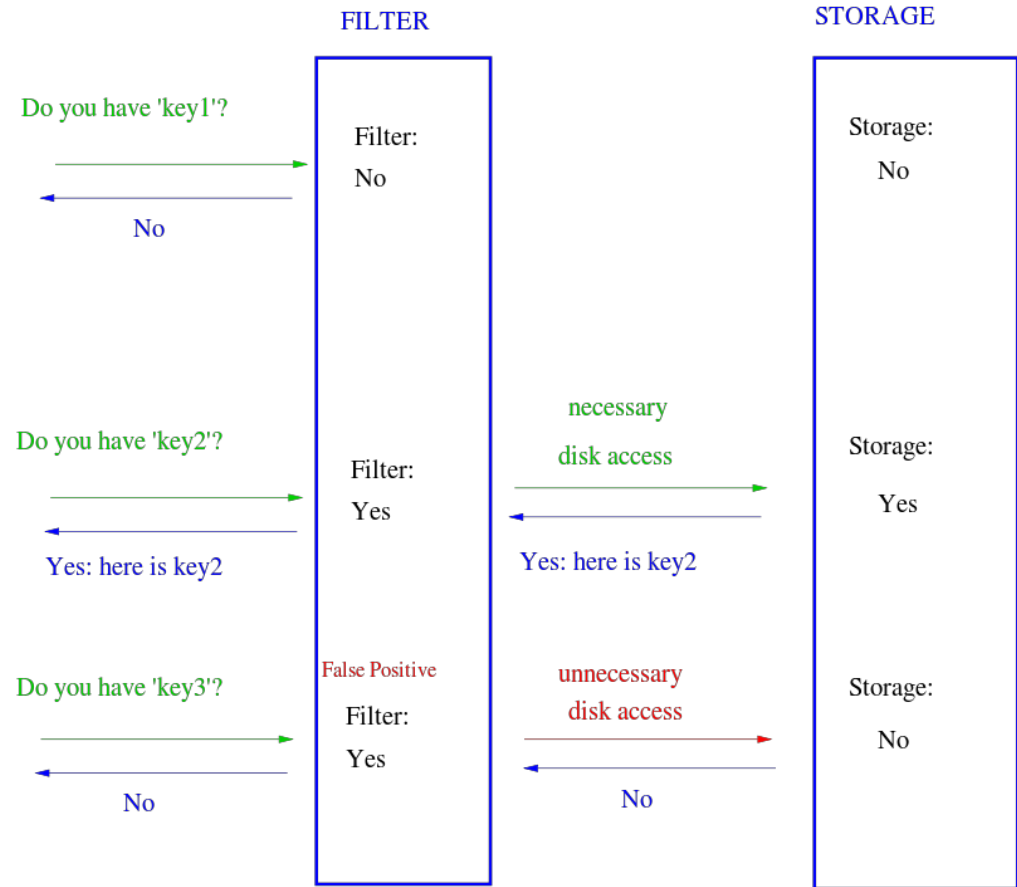
Parameter Settings

- $|S| = 1$ billion, $n = 8$ billion
 - $k = 1$: $(1 - e^{-1/8}) = 0.1175$
 - $k = 2$: $(1 - e^{-1/4})^2 = 0.0493$
- What happens when we keep increasing k to ∞ ?
- Optimal value of k :
 - $k = \frac{n}{m} \ln(2)$
 - Our case, optimal $k = 8 \ln(2) \approx 6$
 - Error at $k = 6$: $(1 - e^{-3/4})^6 = 0.0216$



Storage System

- Bloom filter speeds up answers in a key-value storage system.
- Values are stored on a disk which has slow access times while bloom filter decisions are much faster.



Summary

- Bloom filters guarantee **no false negatives** and use **limited memory**, which is suitable for processing data stream
 - Namely, a query returns either "possibly in set" or "definitely not in set".
 - Great for pre-processing before more expensive check
- Very suitable for hardware implementation
 - Hash function computation can be parallelized

